

هفته ششم: معماری‌های توزیع شده، پیام‌رسانی و کوبرنتیز

مقدمه

در هفته‌های پیشین، تمام تمرکز ما بر ساخت یک اپلیکیشن واحد (Monolith) بود که همگی قابلیت‌ها (API، دیتابیس، کش، احراز هویت) در یک فرآیند اجرا می‌شد. اگرچه این رویکرد برای شروع و پروژه‌های کوچک عالی است، اما برای سیستم‌های بزرگ، با ترافیک بالا و نیازمند مقیاس‌پذیری، چالش‌های جدی پدید می‌آورد. هفته ششم ما را با دنیای معماری میکروسرویس، ارتباط بین سرویس‌ها، صف‌های پیام و در نهایت ارکستراسیون کانتینرها با کوبرنتیز آشنا می‌کند. این دانش، پلی است از یک توسعه‌دهنده بک‌اند به یک مهندس زیرساخت و سیستم‌های توزیع شده.

۱. معماری مونولیتیک در مقابل میکروسرویس

۱.۱ معماری مونولیتیک (Monolithic Architecture)

در این معماری، تمام بخش‌های یک اپلیکیشن (رابط کاربری، منطق کسب‌وکار، دسترسی به داده) در یک کدبیس واحد و یک فرآیند اجرایی قرار دارند. این مانند یک فروشگاه بزرگ است که همه چیز را زیر یک سقف می‌فروشد.

مزایا:

- سادگی در توسعه و دیباگ: برای یک تیم کوچک، همه چیز یکجا است و ردگیری خطاها ساده‌تر است.
- عملکرد بهتر در فراخوانی‌های داخلی: ارتباط بین ماژول‌ها از طریق فراخوانی توابع در حافظه انجام می‌شود و تأخیر شبکه ندارد.
- دیپلوی ساده: فقط یک فایل اجرایی وجود دارد.

معایب:

- مقیاس‌پذیری محدود: اگر فقط یک بخش از برنامه (مثلاً سرویس پرداخت) تحت فشار باشد، باید کل اپلیکیشن را مقیاس دهید.
- وابستگی شدید: یک تغییر کوچک در یک ماژول می‌تواند کل سیستم را مجبور به راه‌اندازی مجدد کند.
- زنجیره فناوری (Tech Stack) قفل‌شده: کل برنامه معمولاً به یک زبان برنامه‌نویسی محدود می‌شود.
- درک دشوار برای توسعه‌دهندگان جدید: یک کدبیس عظیم، یادگیری را سخت می‌کند.

۱.۲ معماری میکروسرویس (Microservices Architecture)

در این معماری، اپلیکیشن به مجموعه‌ای از سرویس‌های کوچک، مستقل و با وظیفه مشخص تقسیم می‌شود. هر سرویس یک قابلیت تجاری خاص را پیاده‌سازی کرده و مالک داده‌ها و منطق خود است. این مانند یک مرکز خرید است که هر مغازه، کسب‌وکاری مجزا با صندوق و انبار مخصوص به خود را اداره می‌کند.

مزایا:

- مقیاس‌پذیری مستقل: فقط سرویس تحت فشار (مثلاً جستجو) را می‌توان مقیاس داد.
- استقلال فناوری: یک سرویس می‌تواند با پایتون و سرویس دیگر با Go نوشته شود.
- تاب‌آوری (Resilience): خرابی یک سرویس، کل سیستم را از کار نمی‌اندازد.
- سرعت توسعه بالاتر: تیم‌های کوچک می‌توانند به صورت موازی روی سرویس‌های مختلف کار کنند.

معایب و چالش‌ها:

- پیچیدگی سیستم توزیع‌شده: مدیریت شبکه، خطاهای گذرا (Transient Failures)، و انسجام داده‌ها بسیار پیچیده می‌شود.
- ارتباطات شبکه‌ای: فراخوانی بین سرویس‌ها کندتر و غیرقابل اعتمادتر از فراخوانی درون‌حافظه‌ای است.

- مدیریت داده‌های توزیع‌شده: پیاده‌سازی تراکنش‌های گسترده (Distributed Transactions) دشوار است و معمولاً از الگوی Saga استفاده می‌شود.
- نیاز به DevOps و ابزارهای پیشرفته: کانتینرها، ارکستراسیون، و مانیتورینگ یکپارچه ضروری است.

۲. نحوه ارتباط بین میکروسرویس‌ها

میکروسرویس‌ها برای همکاری باید با یکدیگر صحبت کنند. دو الگوی اصلی برای این کار وجود دارد:

۲.۱ ارتباط همگام (Synchronous Communication)

- در این روش، یک سرویس درخواستی را به سرویس دیگر ارسال کرده و منتظر پاسخ می‌ماند. مانند یک تماس تلفنی. پروتکل اصلی این کار HTTP/REST یا gRPC است.
- مزایا: ساده و مستقیم؛ برای درخواست‌های فوری که نیاز به پاسخ دارند (مثلاً دریافت اطلاعات کاربر).
 - معایب: ایجاد وابستگی شدید؛ اگر سرویس مقصد از کار بیفتد، درخواست با شکست مواجه می‌شود. این زنجیره وابستگی می‌تواند منجر به "آبشار خطا" (Cascading Failure) شود.

۲.۲ ارتباط ناهمگام (Asynchronous Communication)

- در این روش، سرویس‌ها بدون انتظار برای پاسخ، پیامی را به یک واسط (Message Broker) ارسال می‌کنند. مانند ارسال یک ایمیل. این روش وابستگی را کاهش می‌دهد و برای رویدادهایی که نیاز به اقدام فوری ندارند (مثلاً "کاربر ثبت‌نام کرد") ایده‌آل است. پیاده‌سازی آن معمولاً با صف‌های پیام (Message Queues) یا سیستم‌های Pub/Sub انجام می‌شود.
- مزایا: کاهش وابستگی بین سرویس‌ها، افزایش تاب‌آوری، و توانایی پردازش ترافیک‌های ناگهانی (Spike).
 - معایب: پیچیدگی بیشتر، نیاز به مدیریت واسط پیام و الگوهای پیچیده‌تر مانند Correlation ID برای ردیابی درخواست.

۳. مسیج بروکرها و مفهوم Producer/Consumer

Message Broker یک سرور واسط است که مدیریت ارسال و دریافت پیام بین سرویس‌ها را بر عهده دارد. تولیدکننده (Producer) پیام را به Broker می‌فرستد و مصرف‌کننده (Consumer) پیام را دریافت و پردازش می‌کند. این دقیقاً الگوی ارتباط ناهمگام است که در آن Producer و Consumer از وجود یکدیگر بی‌خبرند.

- Producer (تولیدکننده): سرویسی که یک رویداد (Event) یا دستور (Command) را تولید می‌کند. مثال: سرویس سفارشات، پس از ثبت یک سفارش جدید، رویداد "order_created" را در صف پیام قرار می‌دهد.
- Consumer (مصرف‌کننده): سرویسی که مشترک یک صف یا موضوع (Topic) است و پیام‌ها را دریافت و پردازش می‌کند. مثال: سرویس ارسال ایمیل، با دریافت رویداد "order_created"، یک ایمیل تأیید برای مشتری می‌فرستد.

مزایای این الگو:

- جداسازی (Decoupling): تولیدکننده و مصرف‌کننده می‌توانند به طور مستقل توسعه یافته و مقیاس‌پذیر شوند.
- تراز کردن بار (Load Leveling): حتی اگر مصرف‌کننده به طور موقت کند باشد، پیام‌ها در صف ذخیره می‌شوند تا بعداً پردازش گردند.
- قابلیت اطمینان: پیام‌ها از بین نمی‌روند؛ اگر مصرف‌کننده‌ای دچار مشکل شود، پیام برای پردازش مجدد باقی می‌ماند.

۴. آشنایی با RabbitMQ

RabbitMQ یک Message Broker قدیمی، پایدار و بسیار محبوب است که از پروتکل استاندارد AMQP (Advanced Message Queuing Protocol) پیروی می‌کند. کار با آن ساده و قابل اعتماد است.

مفاهیم کلیدی در RabbitMQ:

- Exchange: نقطه ورود پیام‌ها از Producer. Exchange تصمیم می‌گیرد بر اساس قوانینی (Routing Key)، پیام را به کدام صف‌ها هدایت کند.
- Queue: صفی که پیام‌ها تا زمان مصرف در آن ذخیره می‌شوند.
- Binding: ارتباط بین یک Exchange و یک Queue که بر اساس Routing Key تعریف می‌شود.

الگوهای رایج:

- Work Queue: توزیع وظایف بین چندین Consumer. هر پیام فقط توسط یک Consumer پردازش می‌شود.
- Publish/Subscribe (Fanout): یک پیام برای تمام Consumerهایی که به یک Exchange متصل هستند ارسال می‌شود.
- Topic: ارسال پیام بر اساس الگوهای کلید مسیریابی (مثلاً `order.created`).

RabbitMQ چه زمانی مناسب است؟

وقتی به تحویل دقیقاً یک‌بار (ExactlyOnce Delivery) یا حداقل یک‌بار (AtLeastOnce) نیاز دارید، ترتیب پیام‌ها مهم است، و حجم پیام‌ها در مقیاس فوق‌العاده بزرگ نیست.

۵. آشنایی و کار با Apache Kafka

Kafka یک پلتفرم پخش جریانی توزیع‌شده (Distributed Streaming Platform) است که برای جابجایی حجم عظیمی از داده به صورت بلادرنگ طراحی شده است. برخلاف RabbitMQ که یک صف پیام هوشمند است، Kafka یک لاگ توزیع‌شده (Distributed Commit Log) است.

مفاهیم کلیدی در Kafka:

- Topic: یک کانال یا دسته‌بندی برای پیام‌ها. مانند یک جدول بدون انتها.
- Partition: هر Topic برای مقیاس‌پذیری به چندین پارتیشن تقسیم می‌شود. هر پارتیشن یک فایل لاگ ترتیبی و تغییرناپذیر است.
- Producer: پیام‌ها را در انتهای یک پارتیشن اضافه می‌کند.
- Consumer: پیام‌ها را از یک پارتیشن می‌خواند و اشاره‌گر خود (Offset) را جلو می‌برد.
- Broker: یک سرور Kafka. مجموعه‌ای از Brokerها یک Cluster را تشکیل می‌دهند.

ویژگی‌های منحصربه‌فرد Kafka:

- ماندگاری (Persistence): پیام‌ها بر اساس سیاست‌های تنظیم‌شده (مثلاً به مدت ۷ روز) روی دیسک ذخیره می‌شوند و پس از مصرف نیز حذف نمی‌گردند.
- قابلیت بازپخش (Consumer): Replayها می‌توانند Offset خود را به عقب برگردانند و کل تاریخچه پیام‌ها را مجدداً پردازش کنند. این ویژگی برای تحلیل داده‌های تاریخی و دیباگ بسیار قدرتمند است.
- مقیاس‌پذیری خطی: با اضافه کردن پارتیشن و Broker، توان عملیاتی سیستم به راحتی افزایش می‌یابد.

Kafka چه زمانی مناسب است؟

برای جریان‌های داده با حجم بالا (Big Data Pipelines)، پردازش بلادرنگ (Realtime Analytics)، و معماری‌های Event Sourcing و CQRS که در آن‌ها تاریخچه رویدادها اهمیت دارد.

۶. ساخت Worker و Job در پایتون

برای پردازش وظایف ناهمگام و زمان‌بر در پایتون، از کتابخانه‌هایی مانند Celery استفاده می‌شود. Celery یک صف وظیفه توزیع‌شده است که وظایف را از برنامه اصلی خارج کرده و به Workerها می‌سپارد.

- Task (وظیفه): تابعی که باید به صورت ناهمگام اجرا شود (مثلاً ارسال ایمیل یا فشردن سازی یک تصویر).
- Worker: یک فرآیند جداگانه که وظایف را از صف دریافت و اجرا می‌کند. می‌توان ده‌ها Worker روی یک یا چند ماشین داشت.
- Broker (در Celery): یک Message Broker (مانند Redis یا RabbitMQ) که وظایف بین برنامه اصلی و Workerها را جابجا می‌کند.
- Result Backend: محلی برای ذخیره نتیجه یک وظیفه (اختیاری) که معمولاً Redis یا دیتابیس است.

مزایای استفاده از Worker:

- بهبود زمان پاسخگویی API: کاربر برای انجام کارهای سنگین (مثلاً ارسال هزاران ایمیل) منتظر نمی‌ماند.
- اجرای وظایف زمان‌بندی‌شده (Cron Jobs): می‌توان با Celery Beat وظایفی را در زمان‌های مشخص (مثلاً هر شب ساعت ۳) اجرا کرد.
- مقیاس‌پذیری: می‌توان تعداد Workerها را بر اساس حجم وظایف افزایش داد.

۷. آشنایی با کوبرنتیز (Kubernetes)

Kubernetes (که به اختصار k8s نامیده می‌شود) یک پلتفرم متن‌باز برای ارکستراسیون کانتینرها است. وقتی یک برنامه از ده‌ها یا صدها کانتینر تشکیل شده باشد، مدیریت دستی آن‌ها غیرممکن می‌شود. کوبرنتیز این فرآیند را خودکار می‌کند.

مفاهیم بنیادی:

- Pod: کوچک‌ترین واحد اجرایی در کوبرنتیز. یک Pod می‌تواند یک یا چند کانتینر را که همیشه با هم اجرا می‌شوند، در خود جای دهد.
- Deployment: یک شیء اعلامی (Declarative) که مشخص می‌کند چه تعدادی نمونه (Replica) از یک Pod باید همیشه در حال اجرا باشد. اگر یک Pod از کار بیفتد، Deployment به طور خودکار جایگزینی برای آن ایجاد می‌کند (SelfHealing).
- Service: یک آدرس IP ثابت و نام DNS برای دسترسی به مجموعه‌ای از Podها. از آنجا که Podها موجوداتی فانی هستند و IP آنها دائماً تغییر می‌کند، Service یک نقطه اتصال پایدار فراهم می‌کند و بار را بین Podها توزیع می‌نماید (Load Balancing).
- Ingress: نقطه ورود ترافیک خارجی به کلاستر. Ingress ترافیک HTTP/HTTPS را بر اساس قوانین (مثلاً مسیر URL) به Serviceهای مختلف هدایت می‌کند و معمولاً قابلیت پایان‌دهی (SSL Termination) را نیز دارد.

کوبرنتیز چه مشکلاتی را حل می‌کند؟

- خودترمیمی (SelfHealing): جایگزینی خودکار کانتینرهای خراب.
- مقیاس‌پذیری خودکار (AutoScaling): افزایش/کاهش تعداد Podها بر اساس مصرف CPU یا تعداد درخواست‌ها.
- انتشار تدریجی (Rolling Updates): به‌روزرسانی برنامه بدون قطعی سرویس.
- قابلیت حمل (Portability): یکسان بودن رفتار برنامه روی لپ‌تاپ توسعه‌دهنده، سرورهای داخلی و ابر عمومی.

جمع بندی

هفته ششم دیدگاه ما را از یک اپلیکیشن واحد به یک اکوسیستم توزیع شده گسترش داد. ما یاد گرفتیم که برای غلبه بر محدودیت‌های معماری مونولیتیک، می‌توان از میکروسرویس‌ها استفاده کرد، اما این کار پیچیدگی‌های خود را دارد. ارتباط ناهمگام با استفاده از Message Broker (مانند RabbitMQ برای وظایف و Kafka برای جریان داده‌ها) و Workerها (مانند Celery) کلید مدیریت این پیچیدگی است. در نهایت، برای مدیریت حجم انبوه کانتینرهای میکروسرویس‌ها، با کورنتیز، قدرتمندترین ابزار ارکستراسیون، آشنا شدیم. این مفاهیم، شالوده‌ی کاری یک مهندس نرم‌افزار در مقیاس سازمانی را تشکیل می‌دهند.